

Section 1 — What is the admin side?

Your citizen-facing Flutter app is already complete. Citizens submit pothole and garbage reports, your AI pipeline verifies them, and they get saved to the database. That is the citizen side done.

The admin side is everything that happens after a report is verified. Think of it as the control room that sits on top of your working pipeline. Its job is three things:

- Route each verified complaint to the correct human who can fix it
- Track complaints through a 6-stage pipeline until they are resolved
- Show analytics and intelligence about what is happening across cities

Real-world analogy

Think of the citizen app as a 999 emergency dispatch. The admin side is the control room that receives those calls, assigns officers, tracks response, and measures performance. Without the control room, calls come in but nothing gets done.

What was already done before the admin side

When a citizen submits a report, your existing system already does all of this automatically:

- Layer 0 — checks if the photo is clear and GPS is valid
- Layer 1 — ResNet18 AI classifies the image as pothole or garbage
- Layer 2 — fraud detection checks for GPS spoofing and duplicate reports
- Layer 3 — community verification collects votes from nearby users
- Layer 4 — trust history checks the citizen's reputation score
- Layer 5 — calculates a final confidence score between 0 and 100
- Blockchain Entry 1 — stores the complaint hash on-chain immutably

All of that happens before the admin side even gets involved. The admin side picks up from the moment a report lands in the database with a verified status.

Section 2 — The four actors

The admin system has four actors. Three have accounts. One does not.

Actor 1 — Super Admin

SA Super Admin

Sees all of Pakistan — both cities, both departments, every complaint

Accounts needed: 1 account (you create this once)

Can do:

- ✓ View all complaints across Lahore and Karachi
- ✓ See city-level analytics and risk scores
- ✓ Receive AI escalation alerts when things go wrong
- ✓ Lock or override AI-managed complaint cards
- ✓ Create and manage officer accounts
- ✓ See cross-city comparison dashboard

Cannot do:

- ✗ Directly mark a complaint as resolved
- ✗ Skip the after-photo verification requirement
- ✗ Delete blockchain entries (immutable by design)
- ✗ Delete audit log entries (append-only)

Actor 2 — City Admin

CA City Admin

One per city. Sees everything within their city only. Lahore CA cannot see Karachi data.

Accounts needed: 2 accounts — one for Lahore, one for Karachi

Can do:

- ✓ See all complaints in their city (both departments)
- ✓ View city-level KPI cards and analytics
- ✓ See which department is backlogged
- ✓ Generate PDF reports for their city
- ✓ Receive escalations from their city's department officers

Cannot do:

- ✗ See another city's complaints
- ✗ Directly resolve complaints
- ✗ Skip the two-key resolution requirement

X Create new officer accounts (Super Admin only)

Actor 3 — Department Officer

DO Department Officer

One per department per city. Assigned to exactly one city AND one issue type.

Accounts needed: 4 accounts — Lahore Highways, Lahore LWMC, Karachi Highways, Karachi LWMC

Can do:

- ✓ See complaints assigned to them only
- ✓ Assign complaints to field workers (generates token link)
- ✓ Add notes and updates to complaint cards
- ✓ Upload the after-photo proving the issue is fixed
- ✓ View their own performance analytics

Cannot do:

- X See another officer's complaints
- X See another city's data
- X Click a 'Resolved' button — resolution is automatic
- X Override the 7-day citizen feedback timer

Actor 4 — Field Worker

No account needed — works via tokenized link only

Field workers are the people physically going to fix a pothole or clear garbage. They do NOT get a dashboard login. When a Department Officer assigns a complaint to a field team, the backend generates a single-use tokenized link. The officer shares this link via WhatsApp or SMS. The field worker opens it in their phone browser, uploads the after-photo, and that is it. Token expires after 72 hours and cannot be reused.

Example link format:

```
streetlight.pk/task/SR-9921?token=abc123xyz789
```

The page they see on their phone has three things only: the original citizen photo, a GPS map pin showing exactly where to go, and an Upload After-Photo button. Nothing else. No login. No account. No complexity.

Section 3 — Complete account summary

Here is every account your MVP needs. This is all you seed in your database.

Role	Name (seed data)	City	Department	Sees
Super Admin	Super Admin	All Pakistan	All	Everything
City Admin	Lahore City Admin	Lahore	All depts	All Lahore complaints
City Admin	Karachi City Admin	Karachi	All depts	All Karachi complaints
Dept Officer	Ahmad Raza	Lahore	Highways	Lahore potholes only
Dept Officer	Sara Khan	Lahore	LWMC	Lahore garbage only
Dept Officer	Hassan Malik	Karachi	Highways	Karachi potholes only
Dept Officer	Nadia Iqbal	Karachi	LWMC	Karachi garbage only
Field Workers	No accounts	N/A	N/A	Token link per task only

Total: 7 accounts + unlimited field worker tokens

1 Super Admin + 2 City Admins + 4 Department Officers = 7 seeded accounts. Field workers have zero accounts. The field_worker_tokens table starts empty and fills up automatically as complaints get assigned.

Section 4 — How auto-routing works

Auto-routing is the system that takes a verified complaint and delivers it to the correct officer's Kanban board automatically — with zero human intervention. It uses two inputs and three steps.

The two inputs

Input	Where it comes from	Example value
GPS coordinates	Flutter app captures automatically on submit	lat: 31.521, lng: 74.341
Issue type	Citizen picks from dropdown in the app	pothole or garbage

Step 1 — Detect which city

The backend checks the GPS coordinates against two hardcoded city bounding boxes. If the coordinates fall inside the box, the city is identified.

```
CITIES = {
    "lahore": { lat_min: 31.41, lat_max: 31.63, lng_min: 74.17, lng_max: 74.45 },
    "karachi": { lat_min: 24.74, lat_max: 25.10, lng_min: 66.85, lng_max: 67.20 }
}

# Example: citizen at lat 31.521, lng 74.341
get_city(31.521, 74.341)
# → falls inside lahore box → city = "lahore"
```

Better option: Nominatim reverse geocoding (recommended)

Instead of bounding boxes, use Nominatim (OpenStreetMap's free geocoder). Pass lat/lng, get back the real city name from actual administrative boundaries — not a rectangle approximation. Zero cost, more accurate, handles edge cases correctly. Adding a new city later means adding one line to a dictionary, not calculating coordinates.

Step 2 — Detect which department

A two-entry Python dictionary maps issue type to department. Since you only have potholes and garbage, this is the simplest possible lookup.

```
ISSUE_TO_DEPT = {
    "pothole": "highways",
    "garbage": "lwmc"
}
```

```
# Citizen selected "pothole"
get_dept("pothole")
# → dept = "highways"
```

Pro tip: Validate against AI classification

Your ResNet18 already classifies the image as pothole or garbage. Use this to validate what the citizen selected. If they selected garbage but AI says pothole — override with AI. This catches honest mistakes and reduces misrouting automatically.

Step 3 — Look up the officer

One database query with two filters returns exactly one officer. Your routing_table has 4 rows — one per city/department combination.

```
# routing_table — only 4 rows for the entire MVP
# lahore + highways → Ahmad Raza
# lahore + lwmc     → Sara Khan
# karachi + highways → Hassan Malik
# karachi + lwmc     → Nadia Iqbal

def route_complaint(report, db):
    city = get_city(report.lat, report.lng)
    dept = ISSUE_TO_DEPT[report.issue_type]
    officer = db.query(RoutingTable).filter(
        RoutingTable.city == city,
        RoutingTable.department == dept
    ).first()
    report.assigned_officer_id = officer.id
    report.status = "VERIFIED"
    return officer
```

The routing matrix — all 4 combinations

Department	Lahore	Karachi
Highways (potholes)	Ahmad Raza	Hassan Malik
LWMC (garbage)	Sara Khan	Nadia Iqbal

This table lives in your database as 4 rows. That is the entire routing system. When you want to add Islamabad later, you add 2 more rows and one city entry. The code never changes.

What if GPS is outside both cities?

If a citizen submits from Faisalabad or anywhere outside Lahore and Karachi, the system cannot auto-route. It flags the complaint and notifies the Super Admin.

```
city = get_city(32.081, 72.669) # Faisalabad
# → city = "unknown"

report.status = "FLAGGED"
notify_super_admin("Report outside coverage area.", report)
```

Always handle the unknown case

If you do not handle this, your system will throw an error when a complaint comes from outside your bounding boxes. Always check for 'unknown' city and have a fallback. For MVP this fallback is just flagging it for the Super Admin.

What happens after routing: Email + Dashboard notification

The moment a complaint is verified and routed, two things happen automatically and simultaneously: an email is sent to the assigned officer, and the complaint card appears live on their dashboard. No human triggers either of these — they fire as part of the same routing transaction inside `agent_tools.py`.

Step 4 — Email notification fires to the assigned officer

Immediately after the officer is assigned, a new function `send_routing_email()` in `agent_tools.py` is called. It sends a notification email to the Department Officer containing: complaint ID, issue type, city zone, AI confidence score, community votes, citizen photo thumbnail, and a direct link to the complaint card on the dashboard. The City Admin for that city is CC'd for awareness.

Step 5 — Complaint card appears live on the officer's Kanban dashboard

Because your backend uses Supabase, the React dashboard subscribes to the reports table via Supabase Realtime. The moment `assigned_officer_id` is written, the officer's browser receives the event and the card drops into the VERIFIED column instantly — no page refresh needed. The card carries a yellow “New — Unread” highlight border and the ⚡ lightning bolt badge confirming it was AI-routed. A bell notification icon in the navbar increments by 1, showing the complaint ID, issue type, location, and timestamp when clicked.

Step 6 — Audit log entry written

Every routing action is permanently recorded in `report_logs` with: `changed_by = "agent"`, `previous_status = "NEW"`, `new_status = "VERIFIED"`, `note = "Auto-routed to Ahmad Raza (Lahore Highways). Email sent."`, and `ai_managed = TRUE`. No new tables are needed — only a single boolean column `notification_sent` on the reports table to prevent duplicate emails if the agent loop processes the same complaint twice.

Section 5 — The 6-stage Kanban pipeline

Every complaint moves through exactly six stages. Think of it as a physical board with six columns. Complaint cards move left to right as work gets done.

Stage	What it means	Who moves it	Card colour
1. NEW	Complaint just arrived from AI pipeline	Agentic AI (automatic)	Gray
2. PENDING VERIFICATION	Community votes being collected	Agentic AI (automatic)	Blue
3. VERIFIED	Verified, waiting for officer to assign team	Officer (manual click)	Orange
4. IN PROGRESS	Field team dispatched to fix the issue	Officer or AI escalation	Amber
5. AWAITING FEEDBACK	After-photo uploaded, waiting for citizen	System (automatic)	Purple
6. RESOLVED	Both keys confirmed, complaint closed	System (automatic)	Green

The complaint card — what is on it

Every card on the Kanban board shows the same information:

- Complaint ID — for example SR-9921
- Thumbnail of the citizen's photo
- Issue type badge — POTHOLE or GARBAGE
- AI confidence score — for example 87/100
- Community votes — for example 5 votes
- Reporter trust tier — Rookie, Hero, Paragon etc.
- How long the card has been in the current stage
- Lightning bolt badge if the AI agent moved it
- Lock badge if a human has overridden the AI

The two-key resolution system

No single human can mark a complaint resolved. The system requires two independent confirmations before auto-resolving.

Key	What it is	Who provides it
-----	------------	-----------------

Key 1	Officer uploads after-photo that passes AI 3-check verification	Department Officer
Key 2	Citizen confirms fix via FCM push, OR 7 days pass with no rejection	Citizen (or time)

When both keys are confirmed, the system automatically marks the complaint RESOLVED, writes Blockchain Entry 2 linking to Entry 1, and updates analytics. No button exists for an officer to click Resolved directly.

The 48-hour escalation rule

If a complaint sits in IN PROGRESS for more than 48 hours with no update from the officer, the Agentic AI detects the stall, calls Gemini to write an explanation, and sends an escalation notification to the Super Admin. This forces accountability — officers cannot silently ignore complaints.

```
# Agentic AI detects stall
if hours_in_in_progress > 48:
    note = gemini.reason_and_note(
        f"Stalled {hours}hrs. Officer: {officer.name}.
        Zone: {city}. Issue: {issue_type}."
    )
    escalate_to_super_admin(report, note)
```

Section 6 — The Agentic AI background service

The Agentic AI is a Python background service that runs automatically every 15 minutes. It watches every active complaint and moves cards through the pipeline without requiring any human to press buttons. Think of it as a smart robot manager that never sleeps.

The five files

File	What it does	Uses AI?
complaint_agent.py	Main loop — fetches all active reports and processes each one	No
agent_rules.py	Pure Python decision tree — handles obvious transitions instantly	No
agent_llm.py	Calls Gemini for edge cases and human-readable note writing	Yes (Gemini free tier)
agent_tools.py	The hands — <code>update_status</code> , <code>assign_officer</code> , <code>send_fcm</code> , <code>escalate</code>	No
agent_config.py	All hardcoded thresholds in one place — change without touching logic	No

The observe-reason-act-log loop

For every active complaint, the agent runs through four steps:

1

Observe [System]

Fetch the complaint's current state from the database including status, AI score, GPS, community votes, assigned officer, and how long it has been in the current stage.

2

Reason [Rules Engine]

The pure Python rules engine evaluates the complaint against named rules and returns an action. This handles 90% of cases instantly with zero AI cost. Only ambiguous cases get flagged as `NEEDS_LLM`.

3

Act [Agent Tools]

Execute the decision. This calls one of five tool functions: `update_status`, `assign_officer`, `send_notification`, `escalate`, or `add_note`. Each function does one thing and one thing only.

4

Log [Database]

Write an audit entry to report_logs. Every single action the agent takes is permanently logged with a timestamp, the previous status, the new status, and the reason. This is the accountability trail.

What the rules engine decides

Situation	Rule fires	Action taken	Uses Gemini?
NEW report, score ≥ 70 , GPS valid	Clear case	Move to PENDING_VERIFICATION	No
NEW report, score < 40	Clear case	REJECT, notify citizen	No
PENDING, community votes ≥ 3	Clear case	Move to VERIFIED, assign officer	No
PENDING, Gold/Paragon citizen	Clear case	Move to VERIFIED directly	No
IN_PROGRESS stalled > 48 hours	Ambiguous	Call Gemini for escalation note	Yes
AWAITING_FEEDBACK, 7 days passed	Clear case	Auto-close as RESOLVED	No

Cost breakdown — everything is free

Component	Tool	Monthly cost
Agent scheduler	APScheduler (Python library)	PKR 0
Rules engine	Pure Python logic	PKR 0
LLM reasoning and note writing	Gemini 1.5 Flash (15 RPM free, 1M tokens/day)	PKR 0
Status updates	Supabase (existing database)	PKR 0
Push notifications	Firebase FCM (existing setup)	PKR 0
Audit logging	Supabase report_logs table	PKR 0
TOTAL		PKR 0

Gemini usage estimate: approximately 100 reports per day with only 10% needing the LLM path equals 10 Gemini calls per day. The free tier allows 1,500 calls per day. You are using less than 1% of your free quota.

Section 7 — Analytics dashboard

The analytics dashboard transforms the data your system collects into actionable intelligence. It is not just charts — it predicts problems before they happen and tells officers exactly what action to take.

The two dashboard views

Department Officer view

Sees only their own city and department. One zone tab (non-clickable). Four KPI cards. Donut chart for issue breakdown. Pipeline funnel showing where time is lost. Three AI insight cards. Predictive alerts feed. Generate PDF report button.

City Admin view

Sees their whole city. Can switch between Highways and LWMC tabs. Same KPI cards and charts but across both departments. Can see which department is the bottleneck.

Super Admin view

Sees all of Pakistan. City-wide aggregate KPIs at the top. Two city summary cards side by side — one for Lahore, one for Karachi. Each card shows the City Admin name, risk badge, mini KPIs, and 30-day forecast. Cross-city alerts ranked by severity.

The four KPI cards

KPI card	What it shows	Red alert threshold
Total Reports (30d)	Count of complaints this month + % change vs last month	Spike > 20% above last month
Avg Resolution Time	Average days from submission to RESOLVED	Above 5 days (city target)
Re-report Rate	% of resolved complaints that get reported again	Above 25% means fixes are not holding
Community Score	0-100 composite of citizen engagement and vote quality	Below 40 means verification failing

The three AI insight cards

- Next-30d Forecast — predicts how many complaints to expect next month using a 7-week moving average plus a seasonal index. Lahore pothole complaints spike every monsoon season — this card warns you two weeks early.
- Bottleneck Signal — identifies which pipeline stage has the longest average stall time. If IN_PROGRESS has a 5.1 day average stall, the card shows exactly that with a message: Field team delay suspected.
- Citizen Health Index — a 0-100 score combining returning reporter percentage, average impact scores, verification participation rate, and Paragon citizen count. A score below 50 means citizens are losing trust in the platform.

The predictive alerts feed

Five types of alerts fire automatically based on Python rules running on your data. No paid ML required.

Alert type	When it fires	Example message
Area Warning (red)	Re-report rate above 30%	Re-report rate 31% — field verification recommended
Resource Suggestion (blue)	Pending count above 50 and rising forecast	Assign 2 extra field workers this week
Seasonal Prediction (amber)	Within 3 weeks of historical peak period	Pothole spike expected — monsoon approaching
Department Nudge (orange)	Any category avg resolution above 10 days	Garbage avg 12.4d — target is 5d
Anomaly Alert (red bold)	Single day volume spikes 3x above 30-day average	Lahore volume spiked 3x today — investigate

Section 8 — Complete complaint lifecycle (end to end)

This section walks through the complete journey of a single complaint from citizen submission to blockchain resolution. This is the most important section to understand — everything else in this document serves this flow.

Example: A citizen in Lahore photographs a large pothole near Liberty Chowk and submits it from their phone at 10:00pm on a Monday.

1 Citizen submits report [\[Flutter App\]](#)

The app captures the photo, GPS coordinates (lat: 31.521, lng: 74.341), and the citizen's dropdown selection (pothole). These are sent to your FastAPI backend.

2 AI pipeline runs (Layers 0-5) [\[Existing Backend\]](#)

Your existing verification pipeline processes the report. Image quality check passes. ResNet18 classifies it as pothole with 92% confidence. No GPS spoofing detected. 3 community members verify. Trust score calculated. Final confidence score: 87/100.

3 Blockchain Entry 1 written [\[Solidity Contract\]](#)

The complaint hash, GPS hash, AI score, timestamp, and verification type are stored on-chain. This is your existing ComplaintRegistered function. Entry is permanent and tamper-proof.

4 Agentic AI picks up within 15 minutes [\[complaint_agent.py\]](#)

Rules engine sees: score 87 >= 70, GPS valid. Action: ROUTE. get_city(31.521, 74.341) returns 'lahore'. ISSUE_TO_DEPT['pothole'] returns 'highways'. Routing table lookup returns Ahmad Raza. Report assigned. Status → VERIFIED.

5 Card appears on Ahmad Raza's board [\[React Dashboard\]](#)

Ahmad opens his Kanban board and sees the complaint card in his VERIFIED column. Card shows: SR-9921, large pothole, AI 87/100, 3 votes, Liberty Chowk. Lightning bolt badge shows it was AI-routed.

6 Ahmad assigns field team [\[Department Officer\]](#)

Ahmad clicks Assign to Field Team, selects Field Team B, adds note: 'Ahmad leading — priority fix'. Backend generates tokenized link. Ahmad shares it via WhatsApp to his team leader.

7 Field worker goes to location [\[Field Worker \(mobile link\)\]](#)

Team leader opens the link on their phone. Sees the citizen's photo and the GPS pin. Drives to Liberty Chowk. Fills the pothole. Takes a photo of the repaired road. Taps Upload After-Photo.

8

3-check AI verification runs on after-photo [FastAPI Backend]

Location match check: GPS matches original within 50m. Resolved classifier: ResNet18 confirms road surface is repaired. Timestamp check: after-photo is after submission. All three pass. Token marked as used.

9

FCM push fires to citizen — Key 1 confirmed [Firebase FCM]

Citizen receives a notification: 'Your pothole report at Liberty Chowk has been repaired. Please confirm.' They tap Confirm. Key 2 is now confirmed. If they had not responded in 7 days, the system would auto-close.

10

Both keys confirmed — system auto-resolves [System (automatic)]

Key 1 (verified after-photo) and Key 2 (citizen confirmation) are both present. System sets status to RESOLVED. No human clicked a Resolved button. Analytics dashboard updates. Officer performance metrics log.

11

Blockchain Entry 2 written [Solidity Contract]

New ComplaintResolved entry stored on-chain linking to Entry 1 hash. Contains: officer ID, before/after photo hashes, resolution timestamp, citizen confirmation method. Immutable audit trail complete.

The accountability guarantee

No single actor resolved this complaint alone. The citizen reported it. The AI verified it. The community confirmed it. The officer assigned the team. The field worker fixed it. The AI verified the fix. The citizen confirmed the fix. The blockchain recorded everything. Every step is logged. Nobody can cheat this system without leaving a permanent trace.

Section 11 — Complete list of hardcoded things

This is everything that is hardcoded for MVP. The code is real. Only the data inside it is fake but realistic.

Thing	How many	Where it lives	To add more later
Cities	2 (Lahore, Karachi)	CITIES dict in agent_config.py	Add 1 entry to CITIES dict
City bounding boxes	8 coordinate values	CITIES dict	Add 4 coordinates per city
Departments	2 (Highways, LWMC)	DEPARTMENTS list	Add 1 string
Issue → dept mapping	2 entries	ISSUE_TO_DEPT dict	Add 1 key-value pair
Routing table rows	4 rows	DB seed script	Add 1 row per new city/dept combo
Officer accounts	7 accounts	DB seed script	Create account + add routing row
Issue type dropdown (Flutter)	2 options	Flutter enum	Add 1 option to dropdown

The key insight about hardcoding

The data is hardcoded but the system is fully real. When your supervisor asks whether this scales to more cities, the answer is: yes — add one entry to CITIES, create two officer accounts, add two rows to routing_table. The code never changes. This is the correct way to build an MVP — real architecture, minimal data.

SUMMARY

Component	What it does	Status
4 roles	Super Admin, City Admin, Dept Officer, Field Worker (no account)	Design complete
7 accounts	1 + 2 + 4 seeded officer accounts for MVP	Design complete
2 cities	Lahore and Karachi with GPS bounding boxes	Design complete
2 departments	Highways (potholes) and LWMC (garbage)	Design complete
4 routing rows	Maps every city+dept combination to one officer	Design complete
6-stage Kanban	NEW → PENDING → VERIFIED → IN PROGRESS → AWAITING → RESOLVED	Design complete
Agentic AI	Runs every 15 min, moves cards, Gemini for edge cases, all free	Design complete
Two-key resolution	Officer after-photo + citizen confirmation required	Design complete
Analytics dashboard	KPI cards, charts, AI insight cards, predictive alerts	Design complete
Blockchain Entry 2	Written on resolution, links to Entry 1 hash	Design complete
FCM + SMS + auto-close	3-channel citizen feedback loop	Design complete
Field worker tokens	Single-use 72hr mobile links, no account needed	Design complete